Any additional properties of the user are stored as claims in the AspNetUserClaims table associated with that user. This lets you add arbitrary additional information without having to change the database schema to accommodate it. Want to store the user's date of birth? You could add a claim to that user; there's no need to change the database schema. You'll see this in action in section 23.6, when you add a Name claim to every new user.

**NOTE** Adding claims is often the easiest way to extend the default `IdentityUser`, but you can add properties to the `IdentityUser` directly. This requires database changes but is nevertheless useful in many situations. You can read how to add custom data using this approach here: http://mng.bz/Xd61.

It's important to understand the difference between the `IdentityUser` entity (stored in the AspNetUsers table) and the `ClaimsPrincipal`, which is exposed on `HttpContext .User`. When a user first logs in, an `IdentityUser` is loaded from the database. This entity is combined with additional claims for the user from the AspNetUserClaims table to create a `ClaimsPrincipal`. It's this `ClaimsPrincipal` that is used for authentication and is serialized to the authentication cookie, not the `IdentityUser`.

It's useful to have a mental model of the underlying database schema Identity uses, but in day-to-day work, you shouldn't have to interact with it directly. That's what Identity is for, after all! In the next section we'll look at the other end of the scale: the UI of the app and what you get out of the box with the default UI.

## *23.3.4 Interacting with ASP.NET Core Identity*

You'll want to explore the default UI yourself to get a feel for how the pieces fit together, but in this section I'll highlight what you get out of the box, as well as areas that typically require additional attention right away.